

Logging in Storefront Next

Storefront Next includes a built-in structured logger that outputs JSON to stdout. The logger automatically enriches each log entry with a correlation ID and request context, making requests easy to trace across your application and search in Log Center.

Contents

[Use the Logger](#)[Log Levels](#)[Log Format](#)[View Logs in Log Center](#)[See Also](#)

Use the Logger

Server-Side (Loaders, Actions, Middleware)

Use `getLogger(context)` to get a request-scoped logger. It automatically includes the correlation ID, HTTP method, and path in every log entry.

```
1 import { getLogger } from "@lib/logger.server";
2
3 export async function loader({ context }: LoaderContext) {
4   const logger = getLogger(context);
5   logger.info("Product loaded", { productId: 123 });
6   logger.warn("Inventory low", { remaining: 2 });
7   logger.error("Payment failed", { error: new Error() });
8 }
```

Client-Side or Without Context

Use `createLogger()` when router context isn't available, such as in client-side code or utility functions.

```
1 import { createLogger } from "@lib/logger";
2
3 const logger = createLogger();
4 logger.info("Variant selected", { sku: "var-4" });
```

Metadata

Pass an optional metadata object as the second argument to any log method. Metadata fields are included as JSON properties in the log output.

```
1 import { createLogger } from "@lib/logger";
2
3 const logger = createLogger();
4 logger.info("Variant selected", { sku: "var-4" }, { metadata: { sku: "var-4" } });
```



The logger automatically serializes error instances with `name`, `message`, and `stack` fields.

Log Levels

Storefront Next supports four log levels, from most to least severe:

Level	Use for
<code>error</code>	Unrecoverable failures within a request
<code>warn</code>	Recoverable problems: fallback paths, retries, degraded behavior
<code>info</code>	Observable outcomes: request completed, action succeeded/failed, state changes
<code>debug</code>	Internal progress: intermediate steps, useful for local debugging

Configure the Log Level

On Managed Runtime, configure the log level in your environment settings:

1. Log in to [Runtime Admin](#).
2. Click your project, then your target environment.
3. Click **Environment Settings**.
4. In the Advanced section, set the **Log Level**.
5. Click **Update** and wait for the bundle to redeploy.

Alternatively, set the log level via the Managed Runtime API. Call [projects_target_partial_update](#) with `"log_level"` in the payload.

For local development, use the `SFCC_LOG_LEVEL` environment variable:

```
1 SFCC_LOG_LEVEL=debug npm dev
```

Value	Logs emitted
<code>error</code>	error only
<code>warn</code>	error + warn



debug	all messages
(not set)	warn in production, info otherwise

Log Format

In production, logs are output as structured JSON (ndjson). Each entry contains:

Field	Description
level	Log severity: error, warn, info, or debug
msg	The log message
correlationId	A unique ID for tracing a request across log entries
method	HTTP method (GET, POST, etc.)
path	URL pathname of the request

Example:

```
1 {
2   "level": "info",
3   "correlationId": "a1b2c3d4",
4   "method": "GET",
5   "path": "/products/shoes",
6   "msg": "Product loaded",
7   "productId": "sku-123"
8 }
```

In development, logs are pretty-printed with colors and timestamps for readability.

Correlation ID

Every incoming request is assigned a correlation ID – either extracted from the `x-correlation-id` request header or generated automatically. All log entries for that request share the same `correlationId`, which lets you trace a single request end to end.

View Logs in Log Center

[Log Center](#) lets you search and filter production logs from Managed Runtime and your B2C Commerce instance.



[↗](#). If you don't have the role, ask your Account Manager admin to give you access.

2. Set up your MRT [environment](#):

- Mark the environment as a production environment.
- Select a B2C Commerce instance to connect to the environment.
- Add one or more site IDs to associate with the environment.

Note

MRT logs in Log Center are available only for production environments. There can be a delay of up to 15 minutes between an event and the log appearing in Log Center.

Access MRT Logs

1. [Start Log Center](#) [↗](#).
2. Select a **Region** and [Realm](#) [↗](#).
3. Click **Show filtered results**.
4. Click **Search > Current Search**.
5. Under Service Type, select **mrt**.
6. Under Request Type, select **SSR** for application logs or **CDN** for CDN logs.
7. Under Host, select your production environment (`$PROJECT.$ENVIRONMENT`).

Use the `correlationId` field to filter all log entries for a single request.

Limitations

- Each MRT log message is limited to 32 KB. Characters beyond 32 KB are truncated.
- For information about log levels and log volumes, see [Configuration for Administrators](#) [↗](#).

See Also

- [Build Tools](#)

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)



Try for free



MuleSoft

Tableau

Commerce Cloud

Lightning Design System

Einstein

Quip

Component Library

APIs

Trailhead

Sample Apps

AppExchange

Events and Calend

Partner Communit

Blog

Salesforce Admins

Salesforce Architec

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners.
Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#)

[Use of Cookies](#) [Cookie Preferences](#)  [Your Privacy Choices](#)